

Emergency Dispatch Priority and Coordination System

Final Project Report

ENSE707 Software Quality Assurance

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

Table of Content

1. Project Completion and Final Implementation.....	4
1.1 Project Overview.....	4
1.2 Final Scope and Changes Since Assessment 1.....	4
1.3 Final System Architecture and Design.....	4
1.4 Final Implemented Functionality.....	4
1.5 Final Prototype Evidence.....	4
1.6 Remaining Implementation Limitations.....	4
2. Requirements Verification and Traceability.....	5
2.1 Finalised Requirements.....	5
2.2 Requirements Verification.....	6
2.3 Final Requirements Traceability Matrix.....	6
2.4 Partially Implemented, Unverified and Out-of-Scope Requirements.....	6
3. Test Execution and Final Test Results.....	7
3.1 Final Testing Scope and Approach.....	7
3.2 Final Test Cases.....	7
3.3 Test Execution Results.....	7
3.4 Failed Tests and Corrective Actions.....	7
3.5 Regression Testing.....	7
3.6 Remaining Testing Risks.....	7
4. Test Automation and Quality Engineering.....	8
4.1 Automation Selection.....	8
4.2 Automated Test Implementation.....	8
4.3 Automated Test Results.....	8
4.4 Automation Maintainability.....	8
4.5 Manual and Exploratory Testing.....	8
5. Continuous Integration and DevOps QA.....	9
5.1 Development and Integration Workflow.....	9
5.2 Continuous Integration Pipeline.....	9
5.3 Quality Gates.....	9
5.4 Smoke and Sanity Testing.....	9
5.5 CI Evidence and Limitations.....	9
6. Defect Management and Root Cause Analysis.....	10
6.1 Defect Management Approach.....	10
6.2 Severity, Priority and Status Definitions.....	10

6.3 Defect Lifecycle.....	10
6.4 Final Defect Register.....	10
6.5 Root Cause Analysis: Defect One.....	10
6.6 Root Cause Analysis: Defect Two.....	10
6.7 Root Cause Analysis: Defect Three.....	10
6.8 Corrective and Preventive Actions.....	10
7. Quality Testing.....	11
7.1 Risk-Based Selection of Quality Areas.....	11
7.2 Quality Area One: [e.g. Security].....	11
7.3 Quality Area Two: [e.g. Performance].....	11
7.4 Results and Limitations.....	11
7.5 Recommended Improvements.....	11
8. Quality Metrics and Final Release Decision.....	12
8.1 Final Quality Metrics.....	12
8.2 Requirements and Acceptance-Criteria Evaluation.....	12
8.3 Outstanding Defects and Residual Risks.....	12
8.4 Final Release Decision.....	12
9. AI-Assisted Development and Critical Reflection.....	13
9.1 AI-Assisted Activities.....	13
9.2 Representative GitHub Copilot Evidence.....	13
9.3 Human Review and Validation.....	13
9.4 AI-Related Quality Risks.....	13
9.5 Critical Reflection.....	13
10. Quality Improvement and Future Recommendations.....	14
10.1 Key Quality Lessons.....	14
10.2 Improvements to the Development and QA Process.....	14
10.3 Future Testing and Automation Improvements.....	14
10.4 Future Continuous Quality Engineering.....	14
11. Team Contribution and Professional Reflection.....	15

1. Project Completion and Final Implementation

- ☐ *Complete the software solution or quality improvement prototype proposed in Assessment 1*
- ☐ *Implement the key functional requirements and acceptance criteria*
- ☐ *Document and justify any changes to scope, requirements, architecture, technology, or implementation*
- ☐ *Demonstrate the final system through a working prototype*
- ☐ *Provide a GUI where user interaction is required, or justify an alternative interface (API, CLI, backend service, automated testing framework, or QA dashboard)*

1.1 Project Overview

There is a high volume of calls received by emergency dispatchers every day in New Zealand alone. Sometimes there are too many cases that office workers have to go out into the field to keep up with all of them (1News Reporters, 2026). This causes stress on the current emergency system, which underlines the need for a new response system that can effectively manage all emergency cases and coordinate response units for these cases.

The Emergency Dispatch Priority and Coordination System was developed to improve the management of emergency calls, case handling, prioritisation, department routing, and response-unit coordination. The prototype from phase I allows the dispatcher to create a case and put it through the system, which assigns a priority. The case is then routed to the assigned departments, where they can view the case details on their dashboards with very quick updates. After a unit has finished a case, they can update it on their dashboard, and once all units have signed-off then the case closes.

1.2 Implementation of core requirements and acceptance criteria

In the previous assignment, the functional requirements and non-functional requirements are defined and the acceptance criteria are created to describe expected behaviour of the core functional requirements.

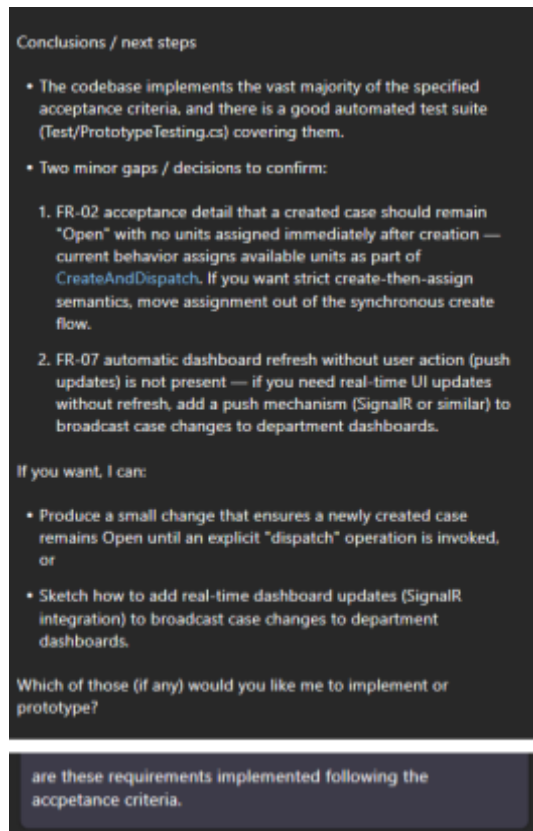


Figure 1.2.1: Github Copilot Evaluation

GitHub Copilot has been used to review the requirements that are gaps identified of core functionality with acceptance criteria. FR-02 and FR-07 are flagged as gaps that need to be considered for further changes. FR-01, FR-04, FR-05, FR-06, FR-08, FR-09 and FR-11 are not found as issues and are considered potential issues by GitHub Copilot. It identified that FR-02 acceptance criteria is not fully met, this criteria in FR-02 “Given a case is newly created from a recorded call, when creation completes, then the case status is set to “Open” with no units assigned to it yet”. FR-07 is flagged because GitHub described that the dashboard does not automatically refresh without user action, which doesn’t meet this: “Given a department’s dashboard is open, when the information of a case changes e.g. priority, location, then the dashboard should refresh and show the new changes automatically.” and “Given a department’s dashboard is open, when a case is routed to that department, then the dashboard should display the new case automatically.”

When personally investigating whether the FR-02 implementation meets the acceptance criteria gaps during testing, there isn’t an issue because if there is a unit available, then the case would be assigned, thus receiving the status of “InProgress”. The acceptance criteria is only met when there are no available units in that relevant department. Therefore, the acceptance criteria would need to be updated that the criteria is only for when there are no available units. For FR-07, after personal testing of the prototype, it has met its expected behaviour since the dashboard is updated without manually refreshing. The suggested GitHub flagged issues because it assumed that automatic updates mean adding a push mechanism (SignalR) to satisfy the stated criteria.

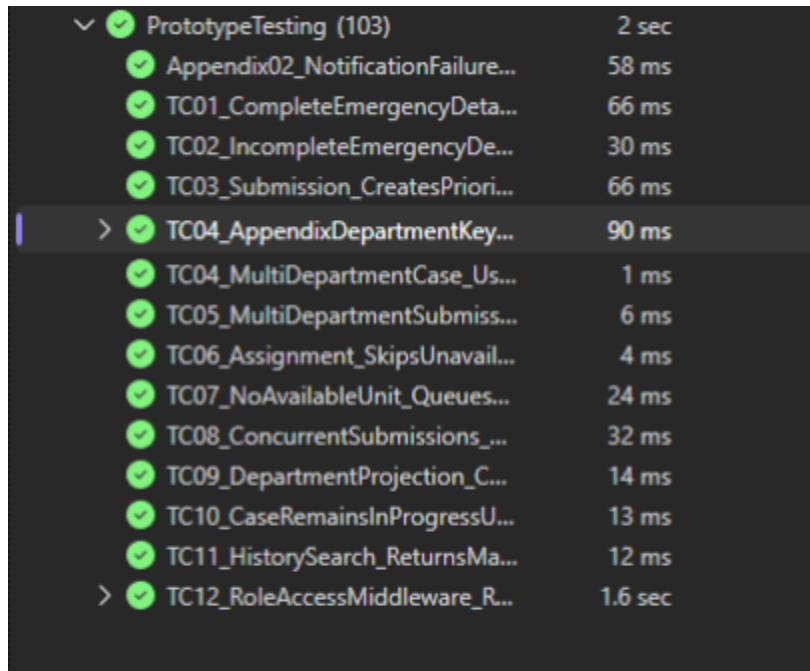


Figure 1.2.2: Testing pass evidence

1.3 Final Scope and Changes Since Assessment 1

The original scope from assignment 1 was to develop an emergency dispatch priority and coordination system prototype that managed emergency cases, case prioritisation, case routing to departments, and response-unit coordination. The scope for the final assignment is the same, but includes some changes to the features due to new findings and corrections required from the previous scope.

Scope Area	Assessment 1 Scope	Final Assignment Scope	Justification
Case recording	Dispatchers can create a case using information from an emergency call.	No changes made. The dispatcher records the caller's details, incident information, location, severity, and the required departments. All of these inputs are validated before case submission	The case-creation scope is unchanged because it provides the necessary information that is used by prioritisation, routing, assignment and testing.
Prioritisation	After submission, the case is put through a	No changes made. The dispatcher submits a case, then the priority system	The prioritisation scope is also the same as assignment 1; more

	priority system which uses keywords to determine its priority.	looks through the case details and uses rules from the case's requested departments, and assigns a priority. When no keywords match, the dispatcher's recommendation is used.	keywords may be added in order to reduce the number of scenarios where keywords don't match the system.
Routing	Submitted cases are routed to departments requested by the dispatcher using in-memory persistence.	The cases in the system are only accessible by the departments that are requested using database operations. For example, a case requesting police and medical should not be accessible by fire	The routing scope remains unchanged, but it was incomplete in assignment 1. Cases should be stored in an emergency service's separate database instead of in-memory arrays.
Unit assignment and coordination	Units are assigned to cases based on availability. If no units are available, the case goes into a queue.	Complete unfinished scope. Units should be assigned to cases based on their availability and distance to the incident. Using both variables, if no units are available, then the case is queued.	The scope for unit assignment did not change; however, it wasn't fully implemented in the assignment 1 prototype. Distance needs to be added as a variable for unit assignment.
Case lifecycle, sign-off, and history	Open, In Progress, and Closed are the three states used to manage active cases. History can be viewed by dispatches, and all assigned units sign off before the case closes.	A case stays Open while waiting for a unit response; it becomes In Progress when a unit is assigned, and Closed when all assigned units sign off. Case assignment must be separate from case creation. Dispatches can view cases in the history tab by caller name, case number or date.	The overall case lifecycle stays the same. But the scope changes to make the case assignment separate from case creation. The history should also be reported as implemented and be described as in-scope.
Role-based GUI	Working dispatcher, response unit, department and admin interfaces. Each displays different tools needed for the user	The final prototype keeps these unique interfaces. Department users are restricted to their own department; so are dispatchers. The admin role is for testing only, so only developers can use it.	There is no change in the Role-based GUI scope; the admin interface should be mentioned as a prototype-only role and is not released in the final delivery.

	role.		
Architecture & Technology	Assignment 1 mentioned using C#, .NET, HTML/CSS, MSTest, and GitHub.	The technology stack is the same as assignment one. The structure of the project, including repository abstractions, priority strategy, the central dispatch service, and separation of different components, will be reported in the final report.	There is no change to the technology stack, however, the architecture was not defined in the report in the previous assignment. This should be highlighted in this assignment.
Security and Persistence	All repositories were in memory, and data was always lost after the program stopped.	The system will use a database file stored in the project folder. This will ensure that developers can restart the program and use previously saved data. Role authorisation is implemented; however, permanent storage, encryption, gps location, and emergency service integration are out of scope.	There is a change in the storage persistence, which will use embedded databases. The security will remain the same, with no encryption of account details or any other data.

1.3 Final System Architecture and Design

1.4 Final Implemented Functionality

1.5 Final Prototype Evidence

1.6 Remaining Implementation Limitations

2. Requirements Verification and Traceability

- ☐ Review and finalise the functional and non-functional requirements from Assessment 1
- ☐ Update the Requirements Traceability Matrix
- ☐ Map requirements to implemented features, test cases, test results, and defects where applicable
- ☐ Identify any requirements that are partially implemented, unverified, or out of scope, and justify them

2.1 Finalised Requirements

From reviewing the 24 finalised requirements in the mid-term report, changes to some of the requirements would need to be made to increase the consistency, testability, clarity, and completeness. These are the requirements that need to be altered to consider the feedback provided in the mid-term report and further evaluation of each requirement:

ID	Original	Finalised	Reason
NFR-SO1	Any data that passes between users and the system must be encrypted to prevent unauthorised access.	The data during transmission between the user and the system shall be encrypted using TLS 1.2 or higher, and the data stored (account data, incident details, and caller data) shall be encrypted using AES-256.	There's no specific testable way to check if the system is encrypted, what counts as preventing unauthorised access, and to what standard. It does not specify the data during transit or data at rest.
NFR-RO1	Data from a case shall not be lost when it is being sent from the dispatcher to the selected emergency department.	The case data shall remain complete and unaltered when the dispatcher transfers data to each selected emergency department.	NFR-RO1 and NFR-RO3 are too similar; NFR-RO1 is about data moving from the dispatcher to the emergency department, and NFR-RO3 is about data being saved. The requirements need to be rephrased in order to provide a more distinct difference.

NFR-RO3	All case log entries and incident reports submitted through the system shall be saved without losing any of the data entered.	All case log entries and incident reports submitted shall be stored persistently, and when retrieving the data, the data to retrieve shall match what was submitted, with no fields missing or changed.	NFR-RO1 and NFR-RO3 are too similar; NFR-RO1 is about data moving from the dispatcher to the emergency department, and NFR-RO3 is about data being saved. The requirements need to be rephrased in order to provide a more distinct difference.
NFR-MO2	The system shall be designed in a way that allows new user roles and departments to be added without requiring modification of existing classes.	The system shall allow new user roles and departments to be created without changing the existing case creations, routing, assignment of cases, or status updates.	This requirement is too strict because modification of existing classes is improbable and could not be confirmed objectively. The new requirement restricts the changes to the core components while removing the restriction for the rest of the components.

The rest of the non-functional requirements and functional requirements were unchanged due to the ambiguity found in the previous improvement iteration of the requirements. All the finalized requirements are in **Appendix 5**.

2.2 Final Requirements Traceability Matrix

2.3 Partially Implemented, Unverified and Out-of-Scope Requirements

3. Test Execution and Final Test Results

- ☐ *Execute the planned tests against the completed system*
- ☐ *Include relevant functional and non-functional tests (unit, integration, system, acceptance, regression, usability, security, performance, or accessibility)*
- ☐ *Include positive, negative, boundary, invalid-input, and error-handling scenarios where applicable*
- ☐ *Provide at least 15 final test cases with execution results*
- ☐ *Summarise the number of planned, executed, passed, failed, blocked, and not-run tests*
- ☐ *Explain failed tests, corrective actions, regression testing, and remaining quality risks*

3.1 Final Testing Scope and Approach

3.2 Final Test Cases

3.3 Test Execution Results

3.4 Failed Tests and Corrective Actions

3.5 Regression Testing

3.6 Remaining Testing Risks

4. Test Automation and Quality Engineering

- ☐ *Identify suitable test cases for automation and justify the selection*
- ☐ *Implement automated tests using an appropriate tool or framework where suitable*
- ☐ *Report automated test execution results using logs, screenshots, reports, or repository evidence*
- ☐ *Discuss the benefits, limitations, and maintainability of the automated test suite*
- ☐ *Where automation is not appropriate, justify the use of manual or exploratory testing*

4.1 Automation Selection

4.2 Automated Test Implementation

4.3 Automated Test Results

4.4 Automation Maintainability

4.5 Manual and Exploratory Testing

5. Continuous Integration and DevOps QA

- ☐ *Describe the development and integration workflow*
- ☐ *Implement a simple CI workflow or equivalent automated quality check where appropriate*
- ☐ *Define quality gates (successful build, passing tests, linting, code review, or coverage threshold)*
- ☐ *Demonstrate smoke or sanity testing where relevant*
- ☐ *If CI/CD is not technically appropriate, justify an alternative quality workflow*

5.1 Development and Integration Workflow

5.2 Continuous Integration Pipeline

5.3 Quality Gates

5.4 Smoke and Sanity Testing

5.5 CI Evidence and Limitations

6. Defect Management and Root Cause Analysis

- ☐ *Maintain a final defect register covering defects found during development and testing*
- ☐ *Define severity, priority, and defect status categories*
- ☐ *Describe the defect lifecycle used in the project*
- ☐ *Analyse at least 3 significant defects using root cause analysis*
- ☐ *Propose corrective actions and defect-prevention measures*
- ☐ *Explain how defect analysis improved product or process quality*

6.1 Defect Management Approach

6.2 Severity, Priority and Status Definitions

6.3 Defect Lifecycle

6.4 Final Defect Register

6.5 Root Cause Analysis: Defect One

6.6 Root Cause Analysis: Defect Two

6.7 Root Cause Analysis: Defect Three

6.8 Corrective and Preventive Actions

7. Quality Testing

- ☐ *Conduct quality testing relevant to the project (security, performance, usability, accessibility, compatibility, reliability, or maintainability)*
- ☐ *Address at least 2 quality areas where applicable*
- ☐ *Use risk-based reasoning to justify the selected testing activities*
- ☐ *Document testing evidence, results, limitations, and recommended improvements*

7.1 Risk-Based Selection of Quality Areas

7.2 Quality Area One: [e.g. Security]

7.3 Quality Area Two: [e.g. Performance]

7.4 Results and Limitations

7.5 Recommended Improvements

8. Quality Metrics and Final Release Decision

- ☐ *Present relevant quality metrics (test pass rate, defect severity distribution, requirement coverage, test coverage, automation coverage, code quality indicators, commit activity)*
- ☐ *Evaluate the completed system against requirements, acceptance criteria, and quality attributes*
- ☐ *Summarise testing results, defect status, metrics, limitations, and residual risks*
- ☐ *State whether the system is ready for release, conditionally ready, or not ready for release*
- ☐ *Justify the release decision using evidence*

8.1 Final Quality Metrics

8.2 Requirements and Acceptance-Criteria Evaluation

8.3 Outstanding Defects and Residual Risks

8.4 Final Release Decision

9. AI-Assisted Development and Critical Reflection

- ☐ *Explain how GitHub Copilot was used (implementation, testing, debugging, refactoring, or QA activities)*
- ☐ *Provide representative evidence of AI-assisted development*
- ☐ *Explain how AI-assisted outputs were reviewed, modified, tested, accepted, or rejected*
- ☐ *Identify quality risks found in AI-assisted outputs (incorrect logic, insecure code, poor maintainability, unsuitable assumptions)*
- ☐ *Reflect on how AI-assisted development affected productivity, learning, testing, software quality, and professional practice*

9.1 AI-Assisted Activities

9.2 Representative GitHub Copilot Evidence

9.3 Human Review and Validation

9.4 AI-Related Quality Risks

9.5 Critical Reflection

10. Quality Improvement and Future Recommendations

- ☐ *Identify the main quality lessons learned from the project*
- ☐ *Recommend improvements to the development and QA process*
- ☐ *Propose future improvements to testing, automation, CI/CD, monitoring, security, usability, accessibility, or maintainability*
- ☐ *Explain how the project could evolve toward stronger continuous quality engineering practice*

10.1 Key Quality Lessons

10.2 Improvements to the Development and QA Process

10.3 Future Testing and Automation Improvements

10.4 Future Continuous Quality Engineering

11. Team Contribution and Professional Reflection

- ☐ *Provide a contribution statement for each group member*
- ☐ *Provide evidence from GitHub commits, project artefacts, testing work, documentation, and implementation tasks*
- ☐ *Include a brief individual reflection from each group member*

11.1 Contribution Statement

11.2 GitHub and Project Evidence

11.3 Individual Reflection – Member One

11.4 Individual Reflection – Member Two

11.5 Individual Reflection – Member Three

Delivarables (Notes)

Prepare a 15-slide presentation covering: title/members/problem/stakeholders/scope, key requirements, final prototype and changes since Assessment 1, testing approach/coverage/results, automation/CI/CD, defects and RCA, quality metrics and release decision, Copilot use and limitations, individual contributions and lessons learned

All students participate in the demonstration and show the working prototype

Demo includes key features, important test cases, automated tests/CI results, defect fixes, evidence major requirements are satisfied

Be prepared to answer questions on the project, code, testing, quality artefacts, AI-assisted development, and individual contributions

Deliverables

- Final Project and Software Quality Assurance Report: 6,000–7,000 words
- Source code and GitHub repository link, with evidence from GitHub commits
- Contribution statement for each group member
- Project presentation
- Turnitin Similarity Report

References:

O Aotearoa, R. I. T. R. I. (n.d.). Pressure grows to fix 111 emergency call system, with 1958 model still in use. RNZ.

https://www.rnz.co.nz/news/emergencies_local/585202/pressure-grows-to-fix-111-emergency-call-system-with-1958-model-still-in-use

1News Reporters. (2026, August 10). St John bracing for another 111 spike amid record demand. 1News.

<https://www.1news.co.nz/2026/08/10/st-john-bracing-for-another-111-spike-amid-record-demand/>